Web Technologies

WET2VO

Teil 7: Formularverarbeitung

Medientechnik und -design | SS 2026

1

Verarbeiten von Formulardaten

GET und POST etwas mehr im Detail

Das Formular

Alternativ: `get`

```
<form method="post" action="<?= $_SERVER["SCRIPT_NAME"] ?>">
  <label for="datenlabel">Info:</label>
  <input type="text" name="daten" id="datenlabel">
  <button type="submit">Abschicken</button>
</form>
```

GET

Werte des Formulars im Query-String übertragen.

Query -String

`?key1=value1&key2=value2`

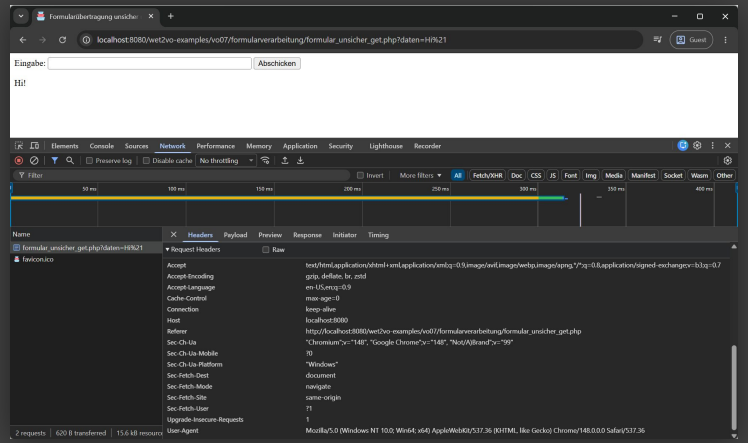
Angehängt an den Request-URL.

```
http://localhost:8080/wet2vo-examples/vo07/formularverarbeitung/
formular_unsicher_get.php
```

↓

```
http://localhost:8080/wet2vo-examples/vo07/formularverarbeitung/
formular_unsicher_get.php?daten=Hi%21
```

Der Request : Keine Daten inkludiert



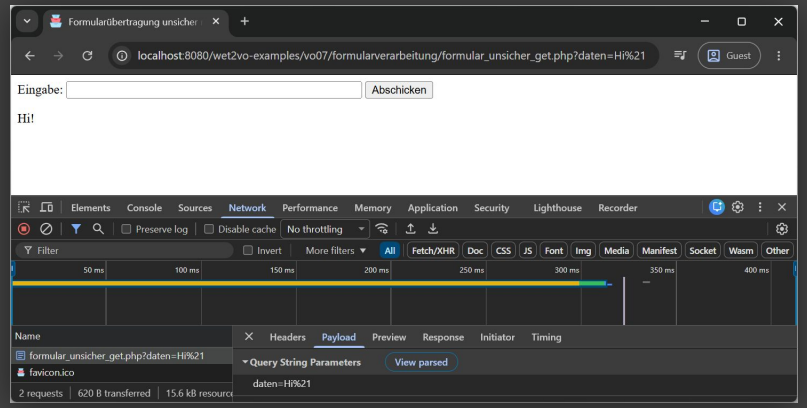
Im Request werden keine Daten oder keine Größenangaben zu den Daten angezeigt.

DevTools -> Network -> Headers

Die Payload : Query-String am URL

Die Payload des Requests wird auch nur als Query-String ausgewiesen.

DevTools -> Network -> Payload



Zugriff: $_GET

$_GET["formfield"]

```
if (isset($_GET["daten"])) {
  echo "<p>{$_GET["daten"]}</p>";
}
```

Unsicher!

POST

Werte des Formulars im Request-Body übertragen.

Request- Body

`key1=value1&key2=value2`

Separat übertragen im Body des Requests (Payload).

```
http://localhost:8080/wet2vo-examples/vo07/formularverarbeitung/
formular_unsicher_post.php
```

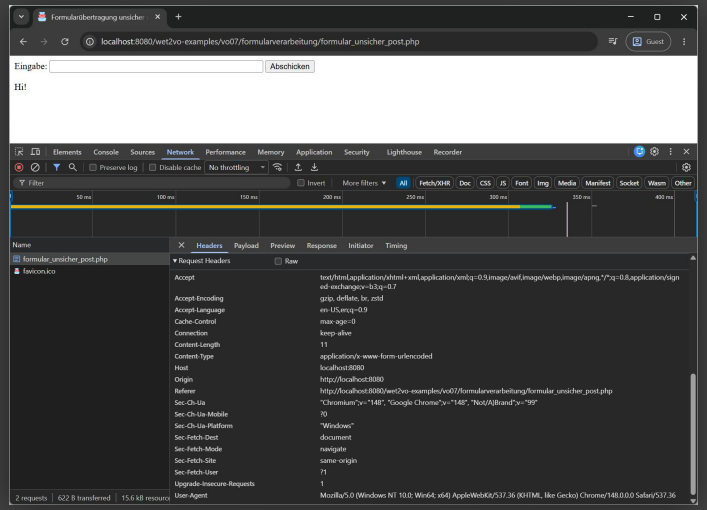
↓

```
http://localhost:8080/wet2vo-examples/vo07/formularverarbeitung/
formular_unsicher_post.php
```

Der Request : Größenangabe als Header

Im Request wird im Header Content-Length die Größe der Daten in Byte angegeben.

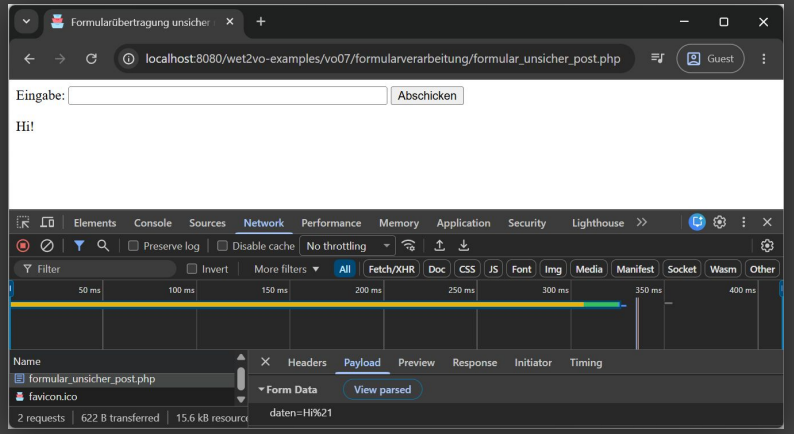
DevTools -> Network -> Headers



Die Payload : Query-String im Request-Body

Die Payload des Requests wird als Query-String im Request-Body übertragen: `daten=Hi%21`

DevTools -> Network -> Payload



Content-Type:

`application/x-www-form-urlencoded`

Zugriff: $_POST

$_POST["formfield"]

```
if (isset($_POST["daten"])) {
  echo "<p>{$_POST["daten"]}</p>";
}
```

GET vs. POST

GET

- Request kann gespeichert und geteilt werden
  (https://somewebshop.site/search?query=someproduct)
- Leichteres Debuggen, da Eingaben im URL sichtbar.
- Zielseite (mit Parametern) kann von Suchmaschinen indiziert werden.

POST

- Formulardaten landen nicht in der History des Browsers.
- Keine Längen- oder Mengenbeschränkung.
- Dateiuploads möglich.
- POST-Requests werden nicht gecached.

2

Gefahren für Webanwendungen

Das Böse ist immer und überall.

XSS

Cross-Site-Scripting

Ungeprüfte Weiterverwendung von Eingaben

XSS: so easy

HTML-Formular:

```
<form method="post" action="<?= $_SERVER["SCRIPT_NAME"] ?>">
  <label for="datenlabel">Info:</label>
  <input type="text" name="daten" id="datenlabel">
  <button type="submit">Abschicken</button>
</form>
```

PHP-Teil:

```
<?php
if (isset($_POST["daten"])) {
  echo $_POST["daten"];
}
?>
```

Was so möglich ist

- Unendliches Neuladen der Seite mittels `location.reload()`,
- Umleitung über `location.href = "https://malicious.site",`
- Auslesen von Cookies über `document.cookie`, wenn nicht durch `HttpOnly`-Flag geschützt.
- Auslesen von localStorage und sessionStorage.
- Auslesen aller gedrückten Tasten über Key-Events und das versenden derselben an eine Zieladresse.

Was so möglich ist

- Verändern der angezeigten Seite (Defacement) oder Einbinden gefälschter Formularfelder (Phishing).
- Unbemerktes Ausführen von Aktionen im Namen des Opfers (Password-Änderungen, Käufe tätigen etc.)

Und viele weitere Dinge, die meist noch viel unerfreulicher sind (Krypto-Miner).

Mehr Gefahren

SQL-Injection

```
https://localhost/mysite/find.php?id=42
SELECT author, subject, text FROM artikel WHERE ID=42

https://localhost/mysite/find.php?id=42;UPDATE+USER+SET+TYPE="admin"+WHERE+ID=23
SELECT author, subject, text FROM artikel WHERE ID=42; UPDATE USER SET
TYPE="admin" WHERE ID=23
```

Cross-Site Request Forgery

```
https://localhost/mysite/user.php?action=new&user=me&pass=secret
```

OWASP

Open Web Application Security Project

https://owasp.org/www-project-top-ten/

OWASP TOP 10

1. A01:2025 - Broken Access Control
2. A02:2025 - Security Misconfiguration
3. A03:2025 - Software Supply Chain Failures
4. A04:2025 - Cryptographic Failures
5. A05:2025 - Injection
6. A06:2025 - Insecure Design
7. A07:2025 - Authentication Failures
8. A08:2025 - Software or Data Integrity Failures
9. A09:2025 - Security Logging and Alerting Failures
10. A10:2025 - Mishandling of Exceptional Conditions

*Keine Benutzer*inneneingaben dürfen in PHP ungefiltert (d. h. ohne entsprechende Überprüfung) wieder ausgegeben werden!*

PHP-Filter-funktionen

Gefährliche Inhalte vor der Weiterverarbeitung entfernen

htmlspecialchars

```
htmlspecialchars($text,
  ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML5)
```

Wandle <, >, &, " und ' in Entitäten um.

```
<h1>"Hällo" 'World'</h1>
          ↓
&lt;h1&gt;&quot;Hällo&quot; &apos;World&apos;&lt;/h1&gt;
```

htmlentities

```
htmlentities($text,
  ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML5)
```

Wandle alle möglichen Zeichen in Entitäten um.

```
<h1>"Hällo" 'World'</h1>
          ↓
&lt;h1&gt;&quot;H&auml;llo&quot; &apos;World&apos;&lt;&sol;h1&gt;
```

filter_var

```
filter_var($text,
  FILTER_SANITIZE_FULL_SPECIAL_CHARS)
```

Entspricht in etwa `htmlentities($text, ENT_QUOTES | ENT_HTML401)`.

```
<h1>"Hällo" 'World'</h1>
          ↓
&lt;h1&gt;&quot;H&auml;llo&quot; &#039;World&#039;&lt;/h1&gt;
```

filter_input

```
filter_input(INPUT_POST, "daten",
  FILTER_SANITIZE_FULL_SPECIAL_CHARS)
```

Wie `filter_var()`, greift jedoch direkt auf z.B. `$_POST["daten"]` zu.

```
<h1>"Hällo" 'World'</h1>
          ↓
&lt;h1&gt;&quot;H&auml;llo&quot; &#039;World&#039;&lt;/h1&gt;
```

strip_tags

`strip_tags($text)`

Entfernt alle HTML-Tags aus dem Text.

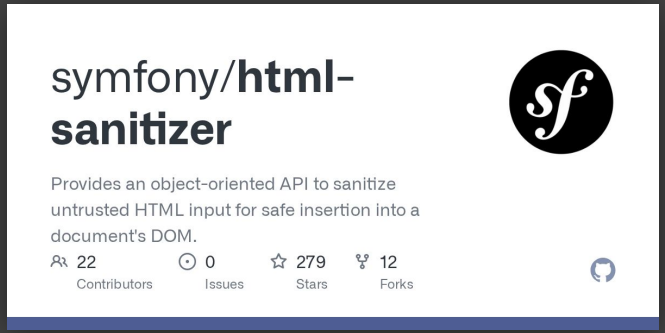
```
<h1>"Hällo" 'World'</h1>
        ↓
"Hällo" 'World'
```

symfony/html-sanitizer

```
$ composer require symfony/html-sanitizer
```

```
$config = new HtmlSanitizerConfig()→allowSafeElements();
$sanitizer = new HtmlSanitizer($config);

if (isset($_POST["daten"])) {
  $datenBereinigt = $sanitizer→sanitize($_POST["daten"]);
  echo "<div>$datenBereinigt</div>";
}
```



3

Dateiuploads

Dateien in Formularen hochladen.

Voraussetzungen

Formular

- Übertragungsmethode POST.
- Content-Type `multipart/formdata`.
- Eingabefeld vom Typ `file`.

Formular- und Servereinstellungen

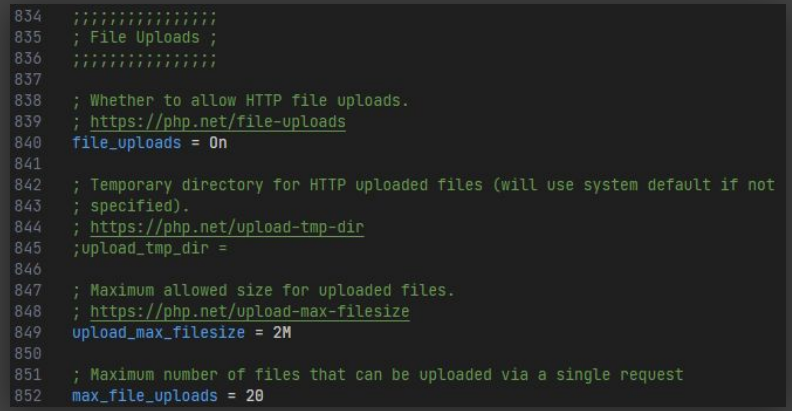
- Soft-Limit für die Größe im Formular.
- Hard-Limit für die Größe in der `php.ini`.

Das Formular

```
<form method="post" enctype="multipart/form-data"
      action="<?= $_SERVER["SCRIPT_NAME"] ?>">
    <input type="hidden" name="MAX_FILE_SIZE"
           value="2097152">
    <input type="file" name="userfile">
    <button type="submit">Hochladen</button>
</form>
```

php.ini
Einstellungen

```
file_uploads = On
upload_max_filesize = 2M
post_max_size = 8M
memory_limit = 128M (-1)
max_execution_time = 30
max_input_time = 60
```



$_FILES

Superglobales Array, das die hochgeladene(n) Datei(en) enthält.

$_FILES im Detail

```
$_FILES["userfile"]["name"]      // sample_image.png
$_FILES["userfile"]["full_path"] // dir/sample_image.png
$_FILES["userfile"]["type"]      // image/png
$_FILES["userfile"]["size"]      // 12761
$_FILES["userfile"]["tmp_name"]  // /tmp/phpV5GBE9
$_FILES["userfile"]["error"]     // 0
```

Fehlercode -Konstanten

0: `UPLOAD_ERR_OK` (Kein Fehler)
1: `UPLOAD_ERR_INI_SIZE` (Angabe in php.ini überschritten)
2: `UPLOAD_ERR_FORM_SIZE` (Angabe im Formular überschritten)
3: `UPLOAD_ERR_PARTIAL` (Datei nur teilweise hochgeladen)
4: `UPLOAD_ERR_NO_FILE` (Keine Datei hochgeladen)
6: `UPLOAD_ERR_NO_TMP_DIR` (Kein Temp-Verzeichnis angegeben)
7: `UPLOAD_ERR_CANT_WRITE` (Keine Schreibrechte)
8: `UPLOAD_ERR_EXTENSION` (Extension hat Upload angehalten)

Fehlercode -Array

```
$uploadErrors = [
  UPLOAD_ERR_INI_SIZE ⇒ "Die Datei überschreitet die in php.ini konfigurierte maximale Größe.",
  UPLOAD_ERR_FORM_SIZE ⇒ "Die Datei überschreitet die im Formular angegebene maximale Größe.",
  UPLOAD_ERR_PARTIAL ⇒ "Die Datei wurde nur teilweise hochgeladen.",
  UPLOAD_ERR_NO_FILE ⇒ "Es wurde keine Datei ausgewählt.",
  UPLOAD_ERR_NO_TMP_DIR ⇒ "Temporäres Verzeichnis fehlt.",
  UPLOAD_ERR_CANT_WRITE ⇒ "Datei konnte nicht auf dem Server gespeichert werden.",
  UPLOAD_ERR_EXTENSION ⇒ "Upload wurde durch eine PHP-Erweiterung abgebrochen.",
];
```

Verschieben der Datei

```php
$error = $_FILES["userfile"]["error"];
if ($error !== UPLOAD_ERR_OK) {
  $message = $uploadErrors[$error] ?? "Unbekannter Fehler (Code $error).";
  echo "<p>Fehler: $message</p>";
} else {
  $originalName = basename($_FILES["userfile"]["name"]);
  if (move_uploaded_file($_FILES["userfile"]["tmp_name"], UPLOAD_DIR . $originalName)) {
    echo "<p>Upload von " . htmlspecialchars($originalName, ENT_QUOTES | ENT_HTML5) .
      "erfolgreich!</p>";
  } else {
    echo "<p>Fehler: Datei konnte nicht gespeichert werden.</p>";
  }
}
```

Upload mehrerer Dateien

Formular:

```
<form method="post" enctype="multipart/form-data" action="<?= $_SERVER["SCRIPT_NAME"] ?>">
    <input type="hidden" name="MAX_FILE_SIZE" value="2097152">
    <input type="file" name="userfiles[]" multiple>
    <button type="submit">Hochladen</button>
</form>
```

Code zum Verarbeiten:

```
$nrOfFiles = count($_FILES["userfiles"]["name"]);
for ($i = 0; $i < $nrOfFiles; $i++) {
  $error = $_FILES["userfiles"]["error"][$i];
  $originalName = basename($_FILES["userfiles"]["name"][$i]);
  $safeName = htmlspecialchars($originalName, ENT_QUOTES | ENT_HTML5);
  if ($error !== UPLOAD_ERR_OK) {
    $message = $uploadErrors[$error] ?? "Unbekannter Fehler (Code $error).";
    echo "<p>Fehler bei $safeName: $message</p>";
  } elseif (move_uploaded_file($_FILES["userfiles"]["tmp_name"][$i], UPLOAD_DIR . $originalName)) {
    echo "<p>Upload von $safeName erfolgreich!</p>";
  } else {
    echo "<p>Fehler: $safeName konnte nicht gespeichert werden.</p>";
  }
}
```

Verzeichnisse hochladen

Formular:

```
<form method="post" enctype="multipart/form-data"
      action="<?= $_SERVER["SCRIPT_NAME"] ?>">
    <input type="hidden" name="MAX_FILE_SIZE" value="2097152">
    <input type="file" name="userfiles[]"
           webkitdirectory multiple>
    <button type="submit">Hochladen</button>
</form>
```

Verzeichnisse hochladen

Code zum Verarbeiten:

```
$nrOfFiles = count($_FILES["userfiles"]["name"]);
for ($i = 0; $i < $nrOfFiles; $i++) {
  $error = $_FILES["userfiles"]["error"][$i];
  $relativePath = $_FILES["userfiles"]["full_path"][$i];
  $destination = UPLOAD_DIR . $relativePath;
  $safePath = htmlspecialchars($relativePath, ENT_QUOTES | ENT_HTML5);


  if ($error !== UPLOAD_ERR_OK) {
    $message = $uploadErrors[$error] ?? "Unbekannter Fehler (Code $error).";
    echo "<p>Fehler bei $safePath: $message</p>";
  } elseif (!is_dir(dirname($destination)) && !mkdir(dirname($destination), 0777, true)) {
    echo "<p>Fehler: Verzeichnis für $safePath konnte nicht erstellt werden.</p>";
  } elseif (move_uploaded_file($_FILES["userfiles"]["tmp_name"][$i], $destination)) {
    echo "<p>Upload von $safePath erfolgreich!</p>";
  } else {
    echo "<p>Fehler: $safePath konnte nicht gespeichert werden.</p>";
  }
}
```

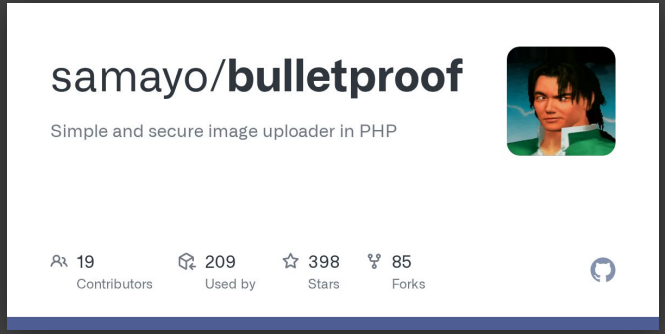
samayo/bulletproof

```
$ composer require samayo/bulletproof
```

```php
$image = new Bulletproof\Image($_FILES);

$image
  ->setStorage("uploads/")
  ->setMime(["jpeg", "png", "gif"])
  ->setSize(1, 2 * 1024 * 1024);

if ($image["userfile"]) {
  $upload = $image->upload();
  if ($upload) {
    echo "<p>Upload von {$upload->getName()} erfolgreich!</p>";
  } else {
    echo "<p>Fehler: {$image->getError()}</p>";
  }
} else {
  echo "<p>Fehler: {$image->getError()}</p>";
}
```



Nächstes Mal in WEF2VO

Sessions und Authentifizierung

Credits

- S. 21: OWASP Top 10 Logo: https://owasp.org/Top10/2025/0x00_2025-Introduction/
- S. 29: symfony/html-sanitizer GitHub Social Image: https://github.com/symfony/html-sanitizer
- S. 42: samayo/bulletproof GitHub Social Image: https://github.com/samayo/bulletproof